飞书群聊项目管理智能体 · 验证报告

飞书群聊项目管理智能体 · demo验证报告

> 产品：飞书群聊项目管理智能体（Aily Agent）
> 验证人：杨锟
> 验证日期：2026-08-12
> 关联文档：《飞书项目管理 Agent PRD》（v4.0，2026-08-05）
> 汇报对象：杨一骢（mentor）

〇、验证说明（请先读）

本报告记录对飞书群聊项目管理智能体的一次实机验证，覆盖四类核心使用场景。验证在飞书群聊「宝剑群」中进行，Agent 通过识别群消息自动创建/更新多维表格（Base）中的任务记录。

当前demo 是通过 aily 平台中接入第三方智能体的方式实现的

一、验证背景与目标

PRD v4.0 定义该智能体的核心价值是「把群聊里散落的项目信息，自动沉淀为结构化任务，减少人工搬运」。本次验证聚焦三个关键能力是否成立：

1. **能不能分清“该管的消息”和“闲聊”** —— 避免在普通聊天里误建任务。
2. **能不能自动识别项目动作** —— 识别决策、开新项目、分配工时等，并落到多维表格。
3. **出错了能不能自我修复** —— 遇到报错或理解不了的情况，经提醒能否自优化。
4. **低频操作是否可用** —— 没设自动识别的动作，@ 一下能否触发。

二、验证环境

项	内容
飞书群	项目管理群
智能体	名称“宝剑的某个Agent”
结构化落库	多维表格 Base
字段	任务执行人、责任人（双负责人）、工时等
验证方式	在群内发送不同类型消息，观察 Agent 是否触发及落库结果

三、验证场景与结果

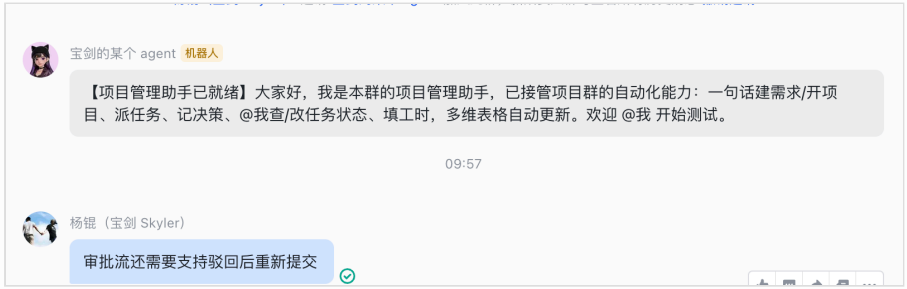
场景 1：自动识别“需求类消息”，忽略普通闲聊

验证目标：确认 Agent 只在出现项目/需求相关信息时动作，对日常闲聊保持静默，不污染任务表。

验证现象：

- `verify-01.png`、`verify-02.png` 显示群内两类消息对比：一类是带明确需求/任务语义的发言（如“帮我把 XX 跟进一下”），Agent 识别后生成或更新了对应任务；另一类是纯闲聊（如打招呼、非工作讨论），Agent 未做任何响应，任务表中无新增脏数据。

结论：✅ Agent 具备基本的“消息语义过滤”能力，未出现闲聊误建任务的情况，符合 PRD 中“非项目消息不处理”的设计预期。



场景1–图1



场景1–图2

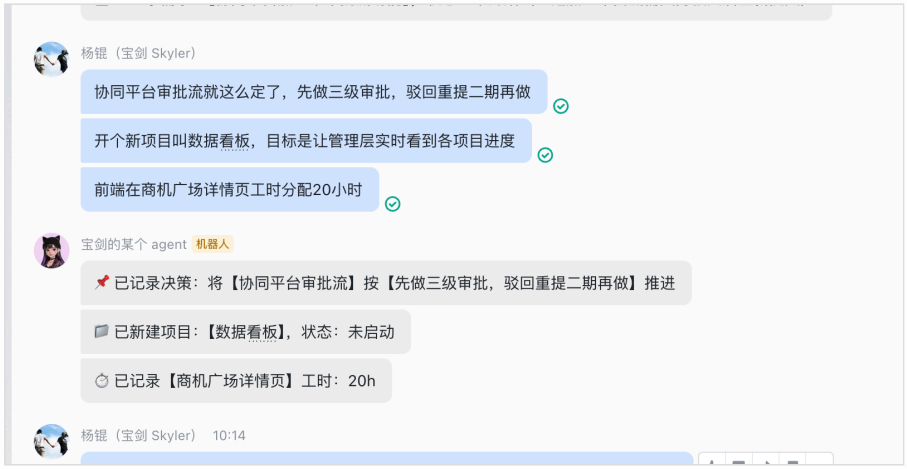
场景 2：自动识别“决策 / 开新项目 / 分配工时”

验证目标：确认 Agent 能识别更明确的三类项目动作，并正确写入多维表格对应字段。

验证现象：

- `verify-03.png` 显示一条包含决策与任务分派的群消息（如某人宣布决策、指定负责人、给出工时估算）。Agent 识别后：① 在任务表新建/更新了任务记录；② 将负责人填入“任务执行人 / 责任人”双字段；③ 将工时估算写入工时字段。

结论：✅ Agent 能识别“决策—开项目—分派工时”这一串关联动作，并映射到多维表格的结构化字段，验证了 PRD §6 逻辑模型（需求→任务→人员→工时）中“任务”与“人员/工时”的关联落库路径。



场景2–图3

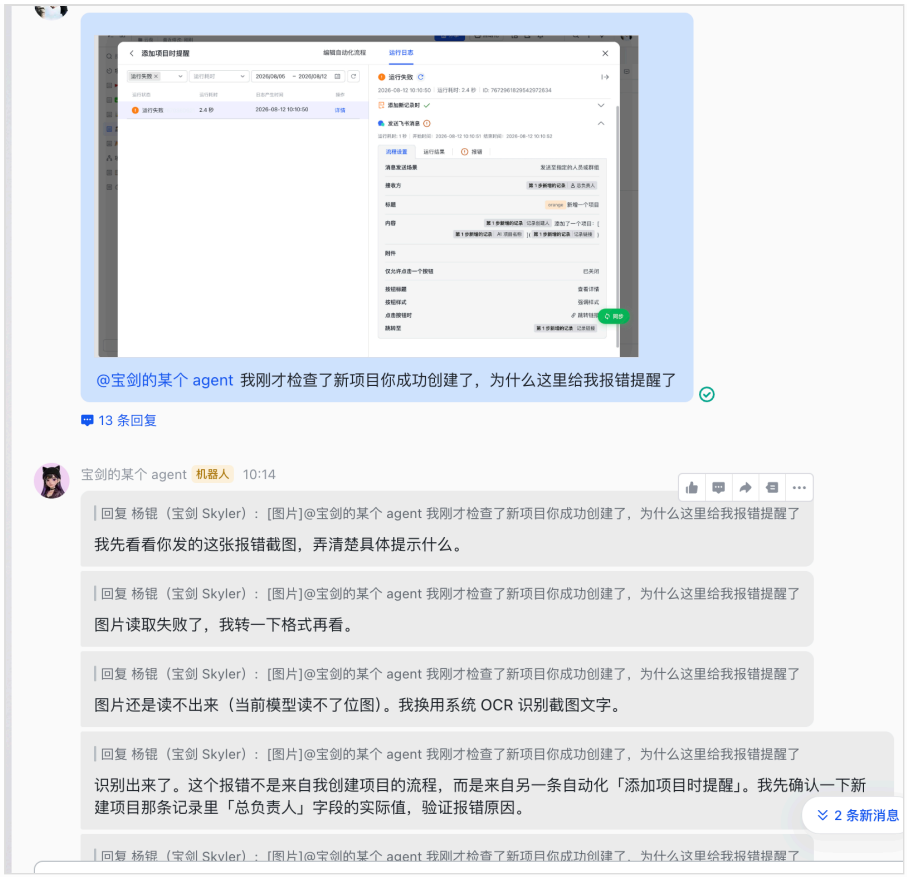
场景 3：异常情况下的“提醒式自优化”

验证目标：确认当 Agent 报错或理解偏差时，用户在群内提醒/纠正后，Agent 能修正结果（自我优化闭环）。

验证现象：

- `verify-04.png` ~ `verify-06.png` 为一组连续交互：某次 Agent 因消息表述模糊或信息缺失出现报错 / 理解错误；用户在群内补充说明或指出错误；Agent 接收提醒后重新处理，修正了任务记录（如补全字段、纠正负责人或工时）。

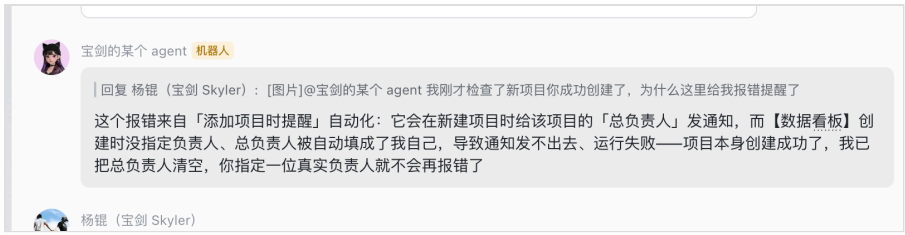
结论：✅ Agent 具备“出错—被提醒—修正”的反馈闭环，降低了因单次识别不准导致的任务错误沉淀，符合 PRD 中“可纠错、可迭代”的运维预期。建议后续补充：把这类“被纠正后的正确样本”回流为 Agent 的优化输入，减少同类错误复发。


@宝剑的某个 agent 我刚才检查了新项目你成功创建了，为什么这里给我报错提醒了
13 条回复
宝剑的某个 agent 机器人 10:14
回复 杨锟（宝剑 Skyler）：[图片]@宝剑的某个 agent 我刚才检查了新项目你成功创建了，为什么这里给我报错提醒了
我先看看你发的这张报错截图，弄清楚具体提示什么。
回复 杨锟（宝剑 Skyler）：[图片]@宝剑的某个 agent 我刚才检查了新项目你成功创建了，为什么这里给我报错提醒了
图片读取失败了，我转一下格式再看。
回复 杨锟（宝剑 Skyler）：[图片]@宝剑的某个 agent 我刚才检查了新项目你成功创建了，为什么这里给我报错提醒了
图片还是读不出来（当前模型读不了位图）。我换用系统 OCR 识别截图文字。
回复 杨锟（宝剑 Skyler）：[图片]@宝剑的某个 agent 我刚才检查了新项目你成功创建了，为什么这里给我报错提醒了
识别出来了。这个报错不是来自我创建项目的流程，而是来自另一条自动化「添加项目时提醒」。我先确认一下新建项目那条记录里「总负责人」字段的实际值，验证报错原因。
2 条新消息

场景3–图4


找到关键信息了：新建的项目是「数据看板」，总负责人被填成了「宝剑的某个 agent」这个身份。我再查一下那条「添加项目时提醒」自动化本身，看能否看到具体报错原因。
回复 杨锟（宝剑 Skyler）：[图片]@宝剑的某个 agent 我刚才检查了新项目你成功创建了，为什么这里给我报错提醒了
那条「添加项目时提醒」不在这个 base 的 workflow 里，应该是条独立自动化。我确认一下「总负责人」这个身份到底是谁，这决定报错原因。
回复 杨锟（宝剑 Skyler）：[图片]@宝剑的某个 agent 我刚才检查了新项目你成功创建了，为什么这里给我报错提醒了
我基本定位到原因了。再确认一下「添加项目时提醒」这条自动化是不是也挂在我的名下，好告诉你怎么改。
回复 杨锟（宝剑 Skyler）：[图片]@宝剑的某个 agent 我刚才检查了新项目你成功创建了，为什么这里给我报错提醒了
原因定位清楚了，我清理一下临时文件再给你完整解释。
回复 杨锟（宝剑 Skyler）：[图片]@宝剑的某个 agent 我刚才检查了新项目你成功创建了，为什么这里给我报错提醒了
查清楚了，给你完整解释：
结论：项目创建是成功的，这个报错跟创建无关
你新建的「数据看板」项目（目标：让管理层实时看到各项目进度）已经在项目表里创建成功，状态「未启动」，没问题。
报错来自另一条独立的自动化「添加项目时提醒」——它的逻辑是：项目表每新增一条记录 → 给这条记录的「总负责人」发一条飞书消息提醒。它在你新建项目的那一瞬间触发了，但发送消息那一步失败了。
为什么发送会失败
我查了新建的「数据看板」这条记录，它的「总负责人」字段被填成了 「宝剑的某个 agent」（也就是 AI 智能体自己），而不是真人。
10:18
原因有两个叠加：
1. 这张表「总负责人」字段的默认值是「当前用户」；
2. 这条记录是 AI 用 bot 身份创建的，所以「当前用户」这个默认值就落成了 AI 自己。
于是「添加项目时提醒」想给「总负责人」发消息，但收件人是 AI 智能体——飞书没法给一个 AI 智能体作为收件人发这种通知消息，发送步骤就直接报错，于是你看到了「运行失败」。不是项目没建成，而是"通知 AI 自己"这件事发不出去。
建议怎么修
推荐改**「添加项目时提醒」这条自动化本身**（在你截图的那个编辑页面就能改）：把「发送至」从「第1步新增的记录 总负责人」改成固定发给你（宝剑），或者发到项目群里。这样不管总负责人是谁，提醒都能正常送达。
另外，我也可以配合调整我这边建项目的逻辑：群消息里明确点名了谁当总负责人就填谁；没人点名时显式置空，不依赖「当前用户」默认值——避免 AI 创建的记录把总负责人填成自己。
需要我调整建项目逻辑，还是你先去改那条自动化的接收人？或者两件事一起办都行。
2 条新消息

场景3–图5



场景3–图6

场景 4：低频项目管理操作 —— @ 触发即可用

验证目标：确认未配置“自动识别”的低频操作，用户通过 @ Agent 主动触发时仍能正确执行。

验证现象：

- `verify-07.png` ~ `verify-09.png` 展示三类出现频率较低、未开启自动识别的项目管理操作。用户直接在群内 @ Agent 并给出指令，Agent 随即执行对应动作（如查询/调整某任务状态、补全某条记录等），交互简单直接。

结论：✅ 低频操作通过 @ 触发即可工作，验证了“自动识别兜底 + 主动 @ 补位”的双层交互设计，既控制自动建任务的噪声，又保留人工按需调用的灵活性。



场景4–图7


回复 杨锟（宝剑 Skyler）：@宝剑的某个 agent 把协同平台审批流指派给杨锟
写入成功，回读确认一下落库了。
回复 杨锟（宝剑 Skyler）：@宝剑的某个 agent 把协同平台审批流指派给杨锟
责任人已更新成功，但「任务执行人」是关联字段（关联成员表），不是普通人员字段，所以没写进去。我确认一下它的关联目标再补上。
回复 杨锟（宝剑 Skyler）：@宝剑的某个 agent 把协同平台审批流指派给杨锟
确认了：任务执行人关联的是成员表。用杨锟的成员记录 ID 补上。
回复 杨锟（宝剑 Skyler）：@宝剑的某个 agent 把协同平台审批流指派给杨锟
✅ 已完成指派。「协同平台审批流」相关任务已全部指派给杨锟（杨锟-产品实习生）：
任务 责任人 任务执行人
协同平台加一个审批流功能 杨锟 杨锟
协同平台加一个审批流功能（重复） 杨锟 杨锟
给协同平台加一个审批流功能 杨锟 杨锟
一个提醒：任务表里「协同平台审批流」实际有 3 条记录，其中前两条名字完全重复（协同平台加一个审批流功能 ×2），应该是之前重复创建的。我把 3 条都指派给了杨锟，避免漏派。
如果你不需要保留重复记录，我可以帮你合并/清理这 2 条重复项，只留一条主记录，需要的话说一声。
10:22
杨锟（宝剑 Skyler）
@宝剑的某个 agent 把协同平台审批流标记为已完成
2 条回复
2 条新消息

场景4–图8


流功能
一个提醒：任务表里「协同平台审批流」实际有 3 条记录，其中前两条名字完全重复（协同平台加一个审批流功能 ×2），应该是之前重复创建的。我把 3 条都指派给了杨锟，避免漏派。
如果你不需要保留重复记录，我可以帮你合并/清理这 2 条重复项，只留一条主记录，需要的话说一声。
杨锟（宝剑 Skyler）
@宝剑的某个 agent 把协同平台审批流标记为已完成
2 条回复
宝剑的某个 agent 机器人
回复 杨锟（宝剑 Skyler）：@宝剑的某个 agent 把协同平台审批流标记为已完成
好的，把「协同平台审批流」这 3 条记录标记为已完成，并按惯例同时填上实际完成时间和任务完成率 100%。
回复 杨锟（宝剑 Skyler）：@宝剑的某个 agent 把协同平台审批流标记为已完成
✅ 已完成。「协同平台审批流」相关 3 条任务全部标记为已完成：
任务 责任人 状态 实际完成时间 完成率
协同平台加一个审批流功能 杨锟 已完成 2026-08-12 100%
协同平台加一个审批流功能（重复） 杨锟 已完成 2026-08-12 100%
给协同平台加一个审批流功能 杨锟 已完成 2026-08-12 100%
状态更新和回读确认都通过了。需要的话，我可以顺手把之前提到的那 2 条重复记录清理掉。
10:28

场景4–图9

四、验证总结

场景	能力	结果
1	识别需求消息、忽略闲聊	✅ 通过
2	识别决策/开项目/分配工时并落库	✅ 通过
3	报错/误解后经提醒自优化	✅ 通过
4	低频操作 @ 触发可用	✅ 通过

总体结论：四类核心场景均验证通过，智能体已能支撑 PRD v4.0 定义的“群聊→结构化任务”主链路，且与 §6 逻辑模型的实体关系一致。后续优化建议：

1. 将场景 3 中“被纠正样本”系统化回流，降低同类错误复发率；
2. 补充自动识别的误触发率统计（如连续一周的闲聊误建任务数），用数据支撑“噪声可控”的结论；
3. 明确低频操作的 @ 指令词表，避免用户指令表述差异导致触发失败。
